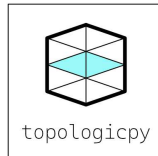


Adding Another Graph Database Backend to TopologicPy



The current design of TopologicPy already makes this relatively straightforward because GraphDB.py is a provider-neutral dispatcher, while Neo4j.py is a concrete backend implementation. To add a new graph database (here we will call it Turbograph), you would mostly add a new Turbograph.py backend class and register it in GraphDB.py.

1. Treat GraphDB.py as the common facade

GraphDB does not store graph data itself. It stores a small descriptor containing the provider name, manager, optional database name, and backend-specific options, then dispatches calls to the selected backend class.

The first conceptual step is to allow a descriptor like this:

```
graphdb = GraphDB.ByParameters(  
    provider="turbograph",  
    manager=manager,  
    database=None,  
    options={}  
)
```

To make this work, GraphDB.py must recognise "turbograph" as a valid provider.

2. Add "turbograph" to the provider aliases

In GraphDB.ByParameters, extend the alias dictionary and supported-provider check.

```
aliases = {  
    "kuzu": "kuzu",  
    "kùzu": "kuzu",  
    "neo4j": "neo4j",  
    "neo": "neo4j",  
    "turbograph": "turbograph",  
    "tg": "turbograph"  
}
```

Change the provider check to include "turbograph":

```
if provider not in ["kuzu", "neo4j", "turbograph"]:
```

3. Add a Turbograph branch to GraphDB._backend

GraphDB._backend currently imports and returns the backend class for each supported provider. Add a third branch for Turbograph.

```
if provider == "turbograph":  
    try:  
        from topologicpy.Turbograph import Turbograph  
    except Exception:  
        try:
```

```

        from Turbograph import Turboraph
    except Exception as e:
        if not silent:
            print(f"GraphDB._backend - Error: Could not import Turbograph: {e}. Returning None.")
        return None
    return Turbograph

```

4. Create a new Turbograph.py backend class

Use Neo4j.py as the closest template because Turbograph is Cypher-based and commonly accessed through the Bolt protocol. Create a class called Turbograph and keep the same static-method style.

At minimum, implement the following methods first:

```

Manager(...)
Connect(...)
Close(...)
Execute(...)
Query(...)
BatchExecute(...)
EnsureSchema(...)
UpsertGraph(...)
GraphByID(...)
GraphsByQuery(...)
DeleteGraph(...)
EmptyDatabase(...)
ListGraphs(...)

```

Then add the GraphRAG/corpus methods:

```

FetchAllPairs(...)
CandidateCountsForLabels(...)
MaxNeighborsForLabel(...)
FindBestExampleForLabel(...)

```

5. Reuse the canonical storage model

Do not invent a new data model for Turbograph. Reuse the same canonical model used by Neo4j so the backends remain interchangeable.

```

(:Graph {
    id,
    label,
    num_nodes,
    num_edges,
    props
})

(:Vertex {
    id,
    graph_id,
    label,
    x,
    y,
    z,
    props
})

(:Vertex)-[:Edge {
    label,
    props,

```

```
graph_id
}]->(:Vertex)
```

6. Start by copying Neo4j.py, then rename surgically

For a first implementation, the fastest route is to copy Neo4j.py to Turbograph.py, rename the class, and then make small backend-specific changes.

Recommended changes:

1. Copy Neo4j.py to Turbograph.py.
2. Rename class Neo4j to class Turbograph.
3. Rename internal calls from Neo4j.Execute(...) to Turbograph.Execute(...).
4. Replace warning/error message prefixes from "Neo4j" to "Turbograph".
5. Keep helper functions such as _json_dumps, _json_loads, _record_to_dict, _records_to_dicts, _value_from_dict, and _python_dictionary.

7. Decide how to connect to Turbograph

Let's assume Turbograph can usually be accessed through the Neo4j Python driver because it supports the Bolt protocol. Therefore, Turbograph.Manager can initially look very similar to Neo4j.Manager.

For a local development instance, the conceptual connection pattern is:

```
manager = Turbograph.Manager(
    url="bolt://localhost:7687",
    username="",
    password=""
)
```

If authentication is enabled:

```
manager = Turbograph.Manager(
    url="bolt://localhost:7687",
    username="Turbograph",
    password="password"
)
```

Then use:

```
graphdb = GraphDB.ByParameters(
    provider="Turbograph",
    manager=manager
)
```

8. Adapt schema creation carefully

Neo4j.EnsureSchema creates Neo4j-style constraints and indexes. For Turbograph, test each statement because not all Neo4j schema syntax is guaranteed to be accepted.

The aim is still the same:

```
Graph.id should be indexed or unique.
Vertex should be efficiently searchable by graph_id and id.
Vertex.label should be searchable.
Edge.graph_id should be searchable where supported.
```

If Turbograph does not support one exact Neo4j schema statement, skip it gracefully rather than failing the whole backend.

9. Keep UpsertGraph behaviour identical

GraphDB.UpsertGraph forwards the TopologicPy graph, mantissa, and silent arguments to the backend implementation. Therefore, Turbograph.UpsertGraph should support the same signature and defaults as Neo4j.UpsertGraph.

```
graphIDKey="id"
vertexIDKey="id"
vertexLabelKey="label"
defaultVertexLabel="Node"
edgeLabelKey="label"
defaultEdgeLabel="CONNECTED_TO"
bidirectional=True
overwrite=False
mantissa=6
silent=False
```

10. Preserve dictionary round-tripping

The backend should preserve TopologicPy dictionaries by serialising them into the props field as JSON when writing, then reconstructing TopologicPy dictionaries when reading.

```
TopologicPy Dictionary -> Python dict -> JSON string -> Turbograph property
Turbograph property -> JSON string -> Python dict -> TopologicPy Dictionary
```

11. Implement GraphByID using the canonical model

Turbograph.GraphByID(manager, graphID, ...) should follow the same reconstruction sequence as Neo4j.GraphByID.

1. Query the Graph node by id.
2. Query all Vertex nodes with graph_id = graphID.
3. Rebuild TopologicPy vertices using x, y, z.
4. Restore vertex dictionaries from props.
5. Query all Edge relationships with the same graph_id.
6. Rebuild TopologicPy edges.
7. Return Graph.ByVerticesEdges(vertices, edges).
8. Attach the graph dictionary.

12. Implement GraphsByQuery as a Cypher result converter

Let's assume Turbograph is Cypher-based. In that case, GraphsByQuery can follow the Neo4j approach. It should run a user query, inspect returned nodes, relationships, and paths, convert nodes to TopologicPy vertices, convert relationships to TopologicPy edges, and return a list containing one TopologicPy graph.

The likely differences are the exact Python object types returned by the driver and whether relationship objects expose exactly the same attributes.

13. Add backend-specific conditionals in GraphDB._call only if necessary

GraphDB._call already has provider-specific handling for Neo4j database names and Kuzu database-name removal. For Turbograph, you probably want to remove database from call_kwargs unless you decide to support a database-like option later.

```
if provider == "Turbograph":
    call_kwargs.pop("database", None)
```

Conceptually, the logic becomes:

```
if provider == "neo4j":
    inject database if provided
```

```
if provider in ["kuzu", "Turbograph"]:  
    remove database
```

14. Test in layers

Test the new backend incrementally rather than trying the whole stack at once.

```
1. Turbograph.Manager(...)
2. Turbograph.Execute(manager, "RETURN 1 AS x")
3. Turbograph.EnsureSchema(manager)
4. GraphDB.ByParameters(provider="Turbograph", manager=manager)
5. GraphDB.EnsureSchema(graphdb)
6. GraphDB.UpsertGraph(graphdb, graph)
7. GraphDB.ListGraphs(graphdb)
8. GraphDB.GraphByID(graphdb, graphID)
9. GraphDB.GraphsByQuery(graphdb, query)
10. GraphRAG methods: CandidateCountsForLabels, MaxNeighborsForLabel, FindBestExampleForLabel
```

Main implementation rule

Make Turbograph.py conform to the same backend contract as Neo4j.py, then register it in GraphDB.py.

The dispatcher should not know Turbograph details. It should only know that the provider name "Turbograph" maps to a class called Turbograph, and that this class implements the same static methods as the other backends.